

Aula 04 — HTTP e o primeiro servidor

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

Duração: 2h · **Pré-requisito:** Aula 03

Objetivos de aprendizagem

Ao final desta aula o aluno deve conseguir:

1. Explicar o que acontece entre digitar um endereço e a página aparecer.
2. Identificar método, caminho, status e corpo em uma requisição HTTP.
3. Criar um servidor com o módulo `http` nativo e entender por que ele é trabalhoso.
4. Criar um servidor com Express e responder em várias rotas.
5. Ler parâmetros de rota (`req.params`) e de query (`req.query`).

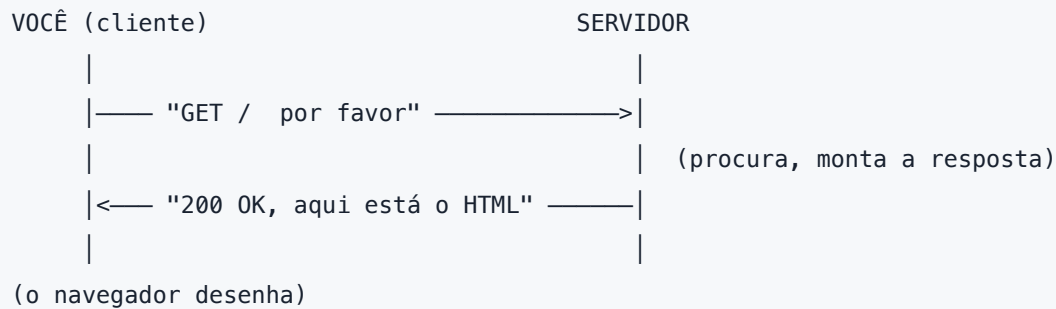
A partir de hoje começamos a construir a **API de Tarefas**, que vai evoluir até a aula 07 e receber uma interface React nas aulas 08 e 09. Guarde o projeto.

Plano de aula (2h)

Tempo	Bloco	Conteúdo
0:00–0:10	Aquecimento	Correção do exercício 4 da aula 03 (paralelo x sequencial)
0:10–0:35	Conceito	Cliente/servidor · anatomia do HTTP · métodos e status
0:35–1:05	Código ao vivo	Servidor com <code>node:http</code> → o mesmo com Express
1:05–1:15	Intervalo	
1:15–1:50	Mão na massa	Exercícios 1 a 3
1:50–2:00	Fechamento	Prévia: transformar isso numa API REST de verdade

1. O que acontece quando você acessa um site

Você digita `google.com` e aperta Enter. Nos milissegundos seguintes:



Isso é o **HTTP**: um protocolo de pergunta e resposta. Simples assim.

Três características que valem entender:

- **O cliente sempre inicia.** O servidor nunca liga para você primeiro; ele só responde.
- **Cada requisição é independente.** O servidor não "lembra" da anterior por conta própria. Isso se chama ser *stateless*.
- **Texto puro.** Uma requisição HTTP é literalmente texto trafegando pela rede.

Analogia: HTTP é como pedir num balcão de lanchonete. Você chega, faz um pedido específico ("um X-burger, sem cebola"), recebe uma resposta (o lanche, ou "acabou o pão"), e a conversa termina. Se quiser mais, faz um novo pedido — e o atendente não precisa se lembrar de você.

Anatomia de uma requisição

```
POST /tarefas HTTP/1.1      ← método + caminho
Host: localhost:3000        ← cabeçalhos (metadados)
Content-Type: application/json

{ "titulo": "Estudar Node" } ← corpo (os dados)
```

Anatomia de uma resposta

```
HTTP/1.1 201 Created        ← status
Content-Type: application/json ← cabeçalhos

{ "id": 1, "titulo": "Estudar Node" } ← corpo
```

2. Os métodos HTTP

O método é o **verbo**: diz o que você quer fazer com aquele recurso.

Método	Intenção	Exemplo
GET	buscar/ler	GET /tarefas — me dê a lista
POST	criar	POST /tarefas — crie esta tarefa
PUT	substituir por inteiro	PUT /tarefas/1 — troque a tarefa 1
PATCH	alterar em parte	PATCH /tarefas/1 — mude só o status
DELETE	remover	DELETE /tarefas/1 — apague a tarefa 1

O navegador, ao digitar uma URL, sempre faz `GET`. Por isso você **não consegue testar um POST pela barra de endereços** — vamos usar outra ferramenta para isso.

3. Os status codes

O status resume o resultado em um número. A primeira casa já diz o essencial:

Faixa	Significado	Como memorizar
2xx	deu certo	"toma aqui"
3xx	redirecionamento	"está em outro lugar"
4xx	erro do cliente	"você errou o pedido"
5xx	erro do servidor	"eu que quebrei, desculpa"

Os que você vai usar toda semana:

Código	Nome	Quando usar
200	OK	deu certo (GET, PUT, PATCH, DELETE)
201	Created	criou algo novo (POST)
204	No Content	deu certo e não há nada para devolver (DELETE)
400	Bad Request	o cliente mandou dados inválidos
401	Unauthorized	não está autenticado
403	Forbidden	está autenticado, mas não pode
404	Not Found	o recurso não existe
500	Internal Server Error	seu código quebrou

A diferença entre 4xx e 5xx importa muito. 4xx é culpa de quem pediu; 5xx é culpa de quem respondeu. Retornar 500 para um dado inválido é esconder um bug do cliente — e vice-versa.

4. Um servidor com o módulo nativo

O Node já traz tudo que é preciso para criar um servidor:

```
import { createServer } from "node:http";

const servidor = createServer((requisicao, resposta) => {
  resposta.writeHead(200, { "Content-Type": "text/plain; charset=utf-8" });
  resposta.end("Olá, mundo!");
});

servidor.listen(3000, () => {
  console.log("Servidor rodando em http://localhost:3000");
});
```

Rode e abra `http://localhost:3000` no navegador. **Você acabou de escrever um servidor web.**

Repare: o programa **não termina**. Ele fica ouvindo a porta 3000, esperando pedidos. Para parar, `Ctrl+C`.

Por que isso não escala

Agora tente ter várias rotas com o módulo nativo:

```
const servidor = createServer((req, res) => {
  if (req.url === "/" && req.method === "GET") { ... }
  else if (req.url === "/tarefas" && req.method === "GET") { ... }
  else if (req.url.startsWith("/tarefas/") && req.method === "GET") {
    const id = req.url.split("/")[2]; // 🤪 partindo string na mão
    ...
  }
  else if (req.url === "/tarefas" && req.method === "POST") {
    // 🤪 para ler o corpo, você precisa juntar pedaços de dados manualmente
  }
});
```

Um `if` gigante, parsing de URL na mão, leitura de corpo por pedaços. Funciona, mas ninguém faz isso em projeto real. Foi exatamente essa dor que criou o Express.

5. Express: o mesmo, sem sofrimento

```
npm install express
```

```
import express from "express";

const app = express();

app.get("/", (req, res) => {
  res.send("Olá, mundo!");
});

app.listen(3000, () => {
  console.log("Servidor rodando em http://localhost:3000");
});
```

Compare com a versão nativa. O Express resolve por você: roteamento por método e caminho, parsing de URL, leitura do corpo, cabeçalhos, status.

As três partes de toda rota

```
app.get("/tarefas", (req, res) => { ... });
// ↑           ↑           ↑   ↑
// |           |           |   | res: o que EU respondo
// |           |           |   | req: o que o cliente PEDIU
// |           |           |   | o caminho
// |           |           |   | o método HTTP
```

`req` e `res` são só nomes por convenção — poderiam ser `pedido` e `resposta`. Mas o mundo inteiro usa `req` e `res`; siga a convenção.

Formas de responder

```
res.send("texto"); // texto ou HTML
res.json({ ok: true }); // JSON (o que mais vamos usar)
res.status(404).json({ erro: "..." }); // status + JSON
res.status(204).end(); // sem corpo nenhum
```

`res.status(...)` devolve o próprio `res`, por isso dá para encadear.

⚠ **Regra de ouro:** cada requisição recebe **uma única resposta**. Se você chamar `res.json()` duas vezes, o Express reclama: `Cannot set headers after they are sent to the client`. A causa quase sempre é um `return` esquecido:

```
if (!tarefa) {  
  return res.status(404).json({ erro: "não achei" }); // ← esse return é obrigatório  
}  
res.json(tarefa);
```

6. Recebendo dados do cliente

`req.params` — parâmetros de rota

Os dois-pontos criam uma parte variável do caminho:

```
app.get("/tarefas/:id", (req, res) => {  
  const { id } = req.params; // desestruturação da aula 02!  
  res.json({ id });  
});
```

GET /tarefas/42 → { "id": "42" }

⚠ **Sempre vem como texto**, mesmo parecendo número. Converta com `Number(id)` antes de comparar com um id numérico — `"42" === 42` é `false`.

`req.query` — o que vem depois do ?

```
app.get("/tarefas", (req, res) => {  
  const { concluida, busca } = req.query;  
  res.json({ concluida, busca });  
});
```

GET /tarefas?concluida=true&busca=node → { "concluida": "true", "busca": "node" }

Também sempre texto — repare que `"true"` é uma **string**, não o booleano `true`.

Quando usar cada um?

- `params` → identifica **qual** recurso: `/tarefas/5`.
- `query` → **modifica** a busca: filtro, ordenação, paginação.

7. Testando a API sem sair do editor

O navegador só faz `GET`. Instale a extensão **REST Client** no VS Code e crie um arquivo `api.http`:

```
### Listar tudo
GET http://localhost:3000/tarefas

### Buscar uma
GET http://localhost:3000/tarefas/1

### Criar (usaremos na aula 05)
POST http://localhost:3000/tarefas
Content-Type: application/json

{ "titulo": "Estudar Express" }
```

Um botão **"Send Request"** aparece acima de cada bloco. É mais simples que o Postman e o arquivo fica versionado junto com o código — ou seja, vira documentação viva da sua API.

8. `node --watch`: pare de reiniciar na mão

Toda vez que você muda o código, o servidor precisa reiniciar. Em vez de `Ctrl+C` + seta para cima

- Enter cinquenta vezes:

```
"scripts": {
  "dev": "node --watch server.js"
}
```

`npm run dev` e pronto: salvou, reiniciou.

9. Ferramentas novas desta aula

O objeto `req` — o que o cliente pediu

Todas essas propriedades são objetos comuns; você acessa com ponto ou desestrutura.

Propriedade	O que é	Exemplo
<code>req.method</code>	o método HTTP	<code>"GET"</code>
<code>req.originalUrl</code>	o caminho completo pedido	<code>"/tarefas?ativo=true"</code>
<code>req.params</code>	os <code>:parametros</code> da rota	<code>{ id: "5" }</code>
<code>req.query</code>	o que vem depois do <code>?</code>	<code>{ ativo: "true" }</code>
<code>req.body</code>	os dados enviados no corpo	(aula 05)

⚠ `params` e `query` sempre entregam texto. Mesmo `/tarefas/42` devolve a string `"42"`.

O objeto `res` — o que você responde

Método	O que faz
<code>res.json(objeto)</code>	converte para JSON e envia — o que mais usamos
<code>res.send(texto)</code>	envia texto ou HTML
<code>res.status(numero)</code>	define o status; não envia nada sozinho
<code>res.end()</code>	encerra a resposta sem corpo

⚠ `res.status(404)` sozinho **não responde**. Ele só define o número; ainda falta o `.json()` ou o `.end()`. Como `res.status()` devolve o próprio `res`, dá para encadear: `res.status(404).json({ erro: "..."}).`

`Number(valor)`

O que faz: converte um valor em número. **Quando usar:** sempre que receber número em forma de texto — e no back-end isso é o tempo todo.

```
Number("42");      // 42
Number("3.14");    // 3.14
Number("abc");     // NaN ← "Not a Number": não deu para converter
Number("");       // 0   ← cuidado com esse caso
```

`Number.isNaN(valor)` e `Number.isInteger(valor)`

```
Number.isNaN(Number("abc")); // true → a conversão falhou
Number.isInteger(42);        // true
Number.isInteger(42.5);      // false
Number.isInteger("42");      // false — é texto, não número
```

Por que `Number.isNaN` existe? Porque `NaN` é o único valor em JavaScript que **não é igual a si mesmo**:

```
NaN === NaN           // false 🤔  
Number.isNaN(NaN)     // true  ✓
```

É por isso que validar id com `if (id === NaN)` nunca funciona — e é o motivo de a função existir.

`.split(separador)`

O que faz: parte um texto em pedaços e devolve um array.

```
"/tarefas/5".split("/");    // [ "", "tarefas", "5" ]  
"2026-04-01".split("-");   // [ "2026", "04", "01" ]
```

⚠ Repare no primeiro item de `"/tarefas/5".split("/")`: é uma string **vazia**, porque o texto começa com a barra. É esse tipo de detalhe que torna o parsing manual de URL tão chato — e é exatamente o trabalho que o Express faz por você.

`app.use(...)` e `app.get/post/patch/delete(...)`

```
app.get("/caminho", funcao);    // responde a UM método e UM caminho  
app.use(funcao);               // roda em TODAS as requisições  
app.use("/prefixo", roteador);  // monta um grupo de rotas (aula 06)
```

⚠ **A ordem importa.** O Express testa as rotas de cima para baixo e usa a primeira que casar. Por isso rotas fixas (`/tarefas/estatisticas`) vêm **antes** de rotas com parâmetro (`/tarefas/:id`) — senão a palavra "estatisticas" seria capturada como um id.

Recapitulação em 3 frases

1. HTTP é pergunta e resposta: método + caminho vão, status + corpo voltam.
2. O Express organiza rotas por método e caminho e cuida do trabalho chato por você.
3. `params` diz **qual** recurso; `query` diz **como** filtrar — e os dois chegam sempre como texto.

Tarefa para a próxima aula

Exercícios 1 a 3 de `exercicios/README.md`.

Na próxima aula

Vamos completar a API de tarefas: criar, atualizar e apagar. É o **CRUD**, o esqueleto de praticamente todo sistema que existe.
